Language

Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50%
на набор курсов:
Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 23. LayoutModifier

28 мая 2023

В этом уроке с помощью LayoutModifier разбираемся, как взаимодействуют между собой родительские и дочерние UI элементы во время Layout фазы.

Изучение LayoutModifier даст нам представление о том, как работают Modifier операторы, которые влияют на размер UI элемента и на то, как он будет расположен - width, padding, offset и т.п.

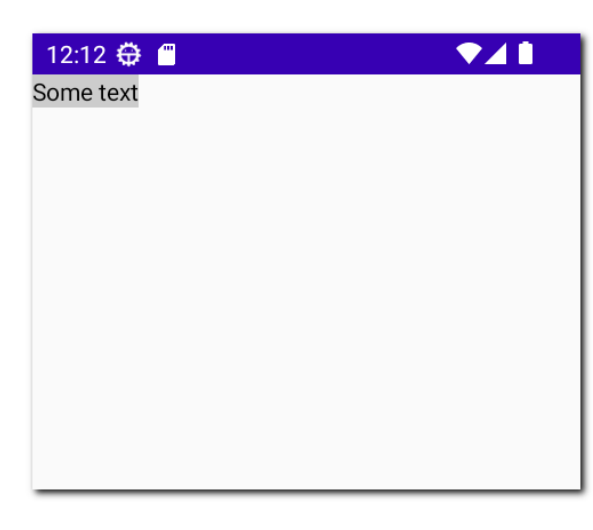
А также это поможет лучше понять следующую тему - Custom layout, потому что у них общий принцип работы.

В качестве примера будем использовать Box, внутри которого расположен Text

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text")
   }
}
```

Чтобы видеть размер Box на экране, я добавил ему серый фон.

Запускаем:

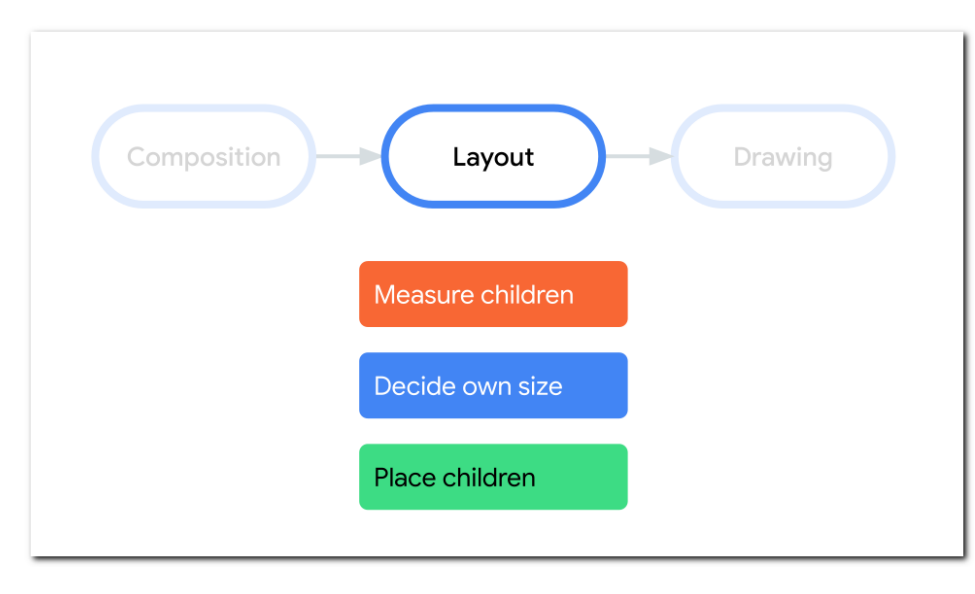


Серый цвет позволяет увидеть, что Box подогнал свои размеры под размеры Text.

Box смог это сделать, потому что под капотом у него с Text произошел примерно следующий диалог:
1) Box спросил у Text, какого размера тот планирует быть
2) Text посмотрел на свой текст и шрифт, все подсчитал и вернул результат, где указаны ширина и высота, которые ему необходимы для отображения текста.
3) Box принял результат и подстроил свои размеры под него.

Если бы внутри Box были еще UI элементы, то он опросил бы их всех. И на основе результатов, вычислил бы свои размеры так, чтобы все элементы влезли.

Этот диалог является важной частью Layout фазы отображения UI



Measure children - родитель опрашивает дочерние UI элементы об их размере
Decide own size - в зависимости от результатов родитель вычисляет собственный размер
Place children - родитель размещает дочерние UI элементы

Это ключевой момент, который необходимо понимать для работы с Custom Layout и LayoutModifier.

LayoutModifier

У нас есть возможность поучаствовать в этом диалоге, создав LayoutModifier с помощью функции layout:

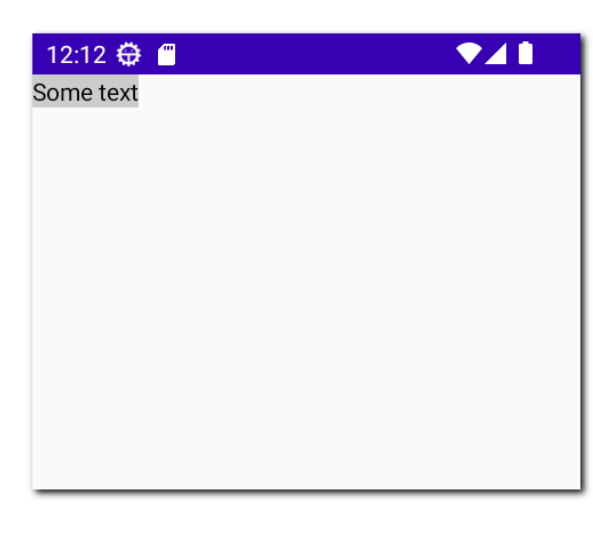
```
fun Modifier.myLayout() = layout { measurable, constraints ->
   val placeable = measurable.measure(constraints) // Measure children
   layout(placeable.width, placeable.height) { // Decide own size
       placeable.placeRelative(0, 0) // Place children
   }
}
```

Мы создаем новый Modifier оператор - myLayout. Он содержит в себе код, который выполняет базовые Layout этапы. Чуть позже мы подробно их разберем.

А пока давайте добавим этот Modifier оператор в Text:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text", modifier = Modifier.myLayout())
   }
}
```

Результат:



Визуально ничего не изменилось. Но под капотом есть одно важное изменение. Изменилась Layout иерархия.

Раньше она была такой:
Box > Text

теперь стала такой:
Box > myLayout > Text

myLayout - это LayoutModifier. Он становится Layout родителем UI элемента, в котором мы его использовали. В нашем случае - он стал родителем для Text, и тем самым вклинился между Box и Text.

Теперь Box будет вести диалог о размерах с myLayout, а myLayout - с Text.

Важно понимать, что myLayout не будет отображаться на экране. Его не найти в Layout Inspector. Это скорее виртуальный промежуточный Layout, который дает нам возможность влиять на диалог между Box и Text. Таким образом мы сможем влиять на размер и расположение Text.

Давайте по шагам разбираться, что происходит внутри нашего myLayout.

constraints

Начнем с constraints, который мы получаем на вход. Пока что просто выведем в лог его значение

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   Log.d(TAG, "constraints = $constraints")
   val placeable = measurable.measure(constraints) // Measure children
   layout(placeable.width, placeable.height) { // Decide own size
       placeable.placeRelative(0, 0) // Place children
   }
}
```

Запускаем и видим следующее:

constraints = Constraints(minWidth = 0, maxWidth = 480, minHeight = 0, maxHeight = 764)

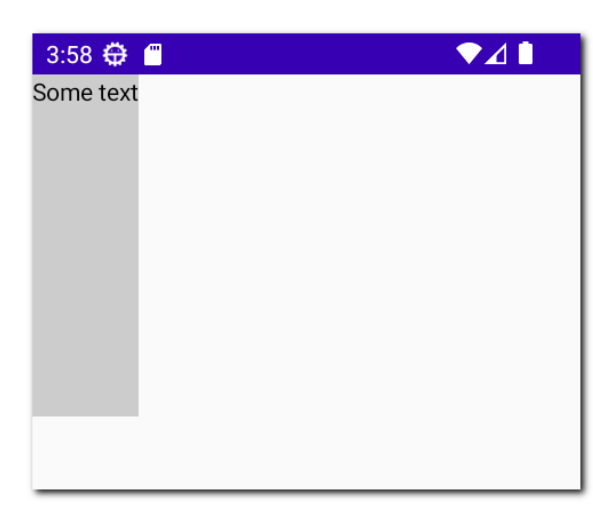
Чтобы понять, что это за данные, вспомним первый пункт диалога: *родитель спрашивает у дочернего элемента, какой размер тот планирует занять*. К этому пункту есть важное дополнение: родитель передает дочернему элементу набор ограничений по размеру, чтобы дочерний учитывал их в своих расчетах. constraints - это и есть те самые ограничения.

В нашем примере myLayout является дочерним элементом для Box. А значит Box спрашивает у myLayout про размер и при этом передает constraints. Мы в HomeScreen не ставили для Box никаких ограничений по размеру, поэтому Box ограничен только размером экрана устройства. Эти ограничения он и передает своим дочерним элементам, т.е. нашему myLayout.
По значениям maxWidth/maxHeight в constraints мы видим, что нам доступен почти весь экран 480x800 px за вычетом высоты статусбара.

Давайте поставим для Box фиксированную высоту в 200.dp и проверим, как это повлияет на ограничения, который Box передает нашему myLayout:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier
           .background(Color.LightGray)
           .height(200.dp)
   ) {
       Text(text = "Some text", modifier = Modifier.myLayout())
   }
}
```

Запускаем



Смотрим лог:

constraints = Constraints(minWidth = 0, maxWidth = 480, minHeight = 0, maxHeight = 300)

Теперь от Box пришел constraints, в котором указано, что максимальная доступная нам высота = 300 px (200.dp). Потому что дочерний элемент не должен быть больше родителя. Если родитель ограничен в размерах, то эти ограничения распространяются и на детей.

px VS dp

Мы в коде указали 200.dp, а в логах получили 300 px. Это не очень удобно, давайте исправим. Будем в коде использовать не dp, а px.

Для этого создадим расширение px:

```
inline val Int.px: Dp
    @Composable get() = with(LocalDensity.current) { this@px.toDp() }
```

Оно переводит px в dp.

Теперь в примерах я смогу указывать значения в px:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier
           .background(Color.LightGray)
           .height(300.px)
   ) {
       Text(text = "Some text", modifier = Modifier.myLayout())
   }
}
```

Это совсем неправильно с точки зрения дизайна, но для тестовых примеров подойдет.

Зато теперь и в коде и в логах у нас одно и то же значение - 300:

constraints = Constraints(minWidth = 0, maxWidth = 480, minHeight = 0, maxHeight = 300)

measurable

Кроме constraints мы в myLayout получаем еще один объект - measurable.

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   ...
}
```

Это средство общения с нашим дочерним элементом, т.е. с Text.

Мы вызываем метод measure, чтобы запросить размеры Text. На вход ему передаем ограничения constraints, полученные от Box, а в качестве результата получаем данные (placeable) о ширине и высоте Text.

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   val placeable = measurable.measure(constraints) // Measure children
   ...
}
```

Наш myLayout пока ничего особо не делает, а просто является промежуточным звеном между Box и Text. Поэтому мы никак не меняем constraints, полученный от Box, а просто передаем его в Text, чтобы тот смог оценить свои размеры.

Чуть позже мы сделаем пример, в котором будем менять constraints, чтобы повлиять на размер Text.

placeable

Как я уже упоминал, placeable - это ответ от дочернего элемента на вопрос родителя: "*Какого размера ты планируешь быть?*". Он содержит данные о планируемой ширине и высоте дочернего элемента.

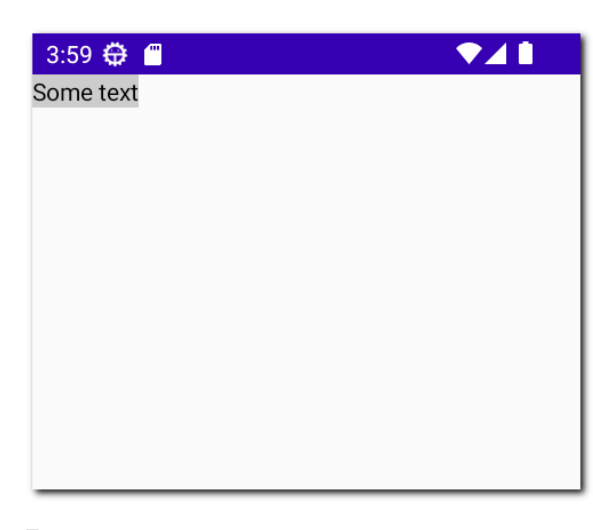
Давайте выведем эти данные в лог:

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   val placeable = measurable.measure(constraints) // Measure children
   Log.d(TAG, "placeable, width = ${placeable.width}, height = ${placeable.height} ")
   layout(placeable.width, placeable.height) { // Decide own size
       placeable.placeRelative(0, 0) // Place children
   }
}
```

И очистим Box от ограничений:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text", modifier = Modifier.myLayout())
   }
}
```

Результат:



Лог:

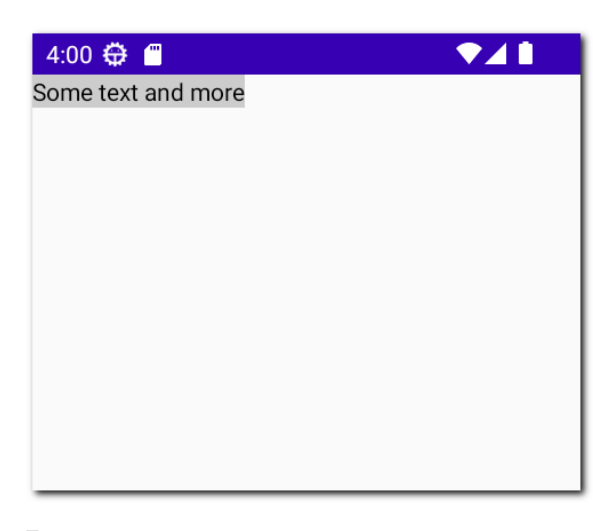
placeable, width = 93, height = 29

Text сделал замеры и сообщил нам, что он планирует быть размером 93x29.

Давайте попробуем добавить в Text еще пару слов и посмотрим, к чему это приведет

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text and more", modifier = Modifier.myLayout())
   }
}
```

Результат:



Лог:

placeable, width = 186, height = 29

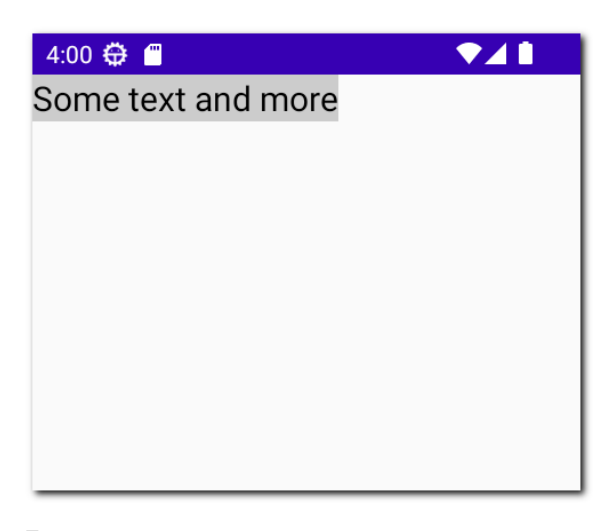
Высота не поменялась, а ширина стала больше, т.к. стало больше текста. Все логично.

А Box подстраивается под новый размер Text, потому что получает эти данные от myLayout. Об этом чуть позже.

Размер шрифта также должен поменять итоговый размер Text. Поставим 20.sp:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text and more", fontSize = 20.sp, modifier = Modifier.myLayout())
   }
}
```

Результат:



Лог:

placeable, width = 268, height = 41

Теперь увеличилась не только ширина, но и высота, т.к. буквы стали больше.

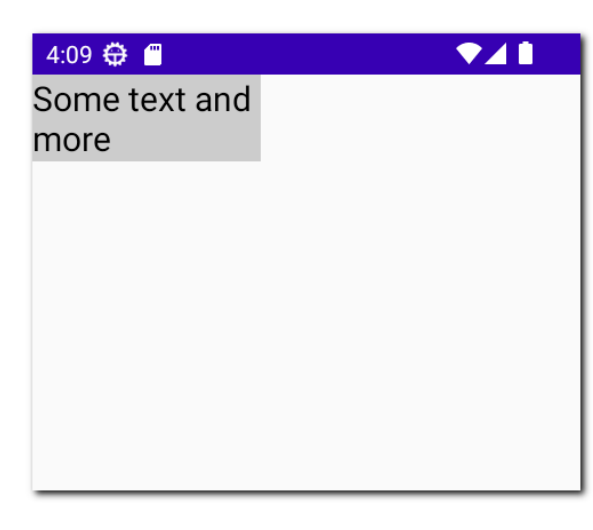
Text с такими параметрами занимает 268 px в ширину. А что будет, если мы попытаемся его ограничить?

Поставим для Box ширину 200 px:

```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier
           .background(Color.LightGray)
           .width(200.px)
   ) {
       Text(text = "Some text and more", fontSize = 20.sp, modifier = Modifier.myLayout())
   }
}
```

Box передаст это ограничение (constraints) нам в myLayout, а мы передадим его тексту. Текст вынужден будет учитывать, что ему доступно только 200 px в ширину.

Ему придется перенести часть текста на следующую строку:



Он нам об этом сообщил в placeable:

placeable, width = 200, height = 76

Ширина - 200, а высота стала почти в два раза больше из-за второй строки.

myLayout передал эти данные в Box, который подстроил свой размер.

layout

Я уже несколько раз упоминал, что Box получает от myLayout размеры и подстраивается под них. Это происходит с помощью функции layout:

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   ...
   layout(placeable.width, placeable.height) { // Decide own size
       ...
   }
}
```

Т.е. myLayout сообщает в Box, какие размеры планирует занимать. Но так как наш myLayout ничего особенного не делает, то он просто передает туда размеры Text. Box получает планируемые размеры Text и ставит себе такие же размеры. В итоге мы на экране видим, как размер Box точно совпадает с размером Text.

Попробуем вмешаться в этот процесс и исказим данные.

Приведем пример к начальному состоянию:

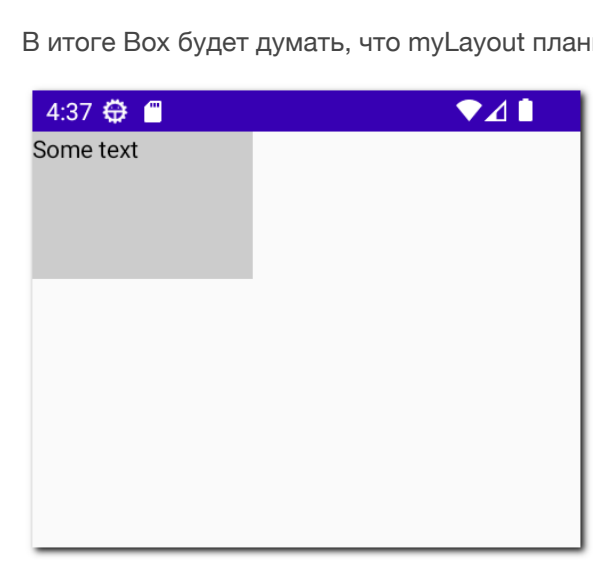
```
@Composable
fun HomeScreen() {
   Box(
       modifier = Modifier.background(Color.LightGray)
   ) {
       Text(text = "Some text", modifier = Modifier.myLayout())
   }
}
```

К ширине и высоте текста будем прибавлять 100:

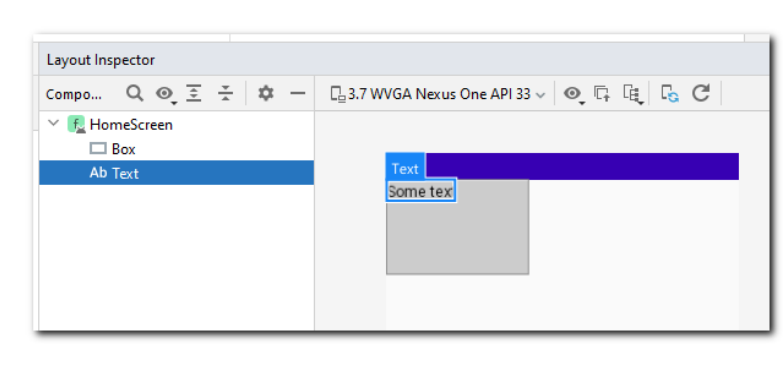
```
fun Modifier.myLayout() = layout { measurable, constraints ->
   val placeable = measurable.measure(constraints) // Measure children
   layout(placeable.width + 100, placeable.height + 100) { // Decide own size
       placeable.placeRelative(0, 0) // Place children
   }
}
```

Т.е. текст нам в placeable вернет width = 93, height = 29, а мы в Box передадим 193 и 129.

В итоге Box будет думать, что myLayout планирует быть размером 193x129 и подстроится под этот размер:



А текст остался прежним - 93x29. Это можно увидеть в layout inspector:



Таким образом мы с помощью myLayout создали отступы в 100 px справа и снизу от текста.

placeRelative

Осталось разобраться в последней строке нашего myLayout:

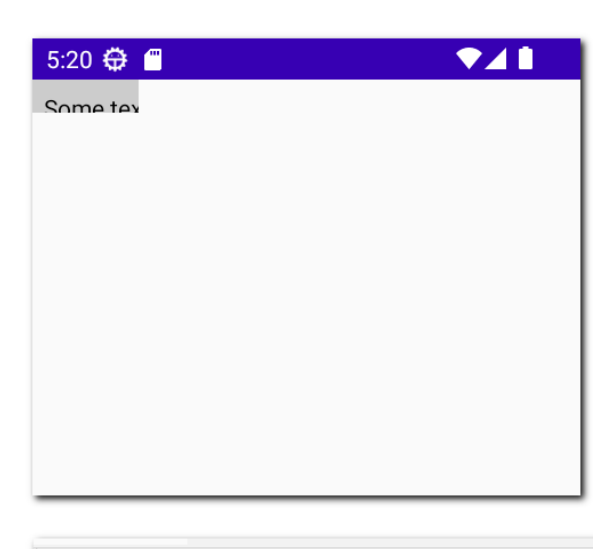
```
fun Modifier.myLayout() = layout { measurable, constraints ->
   ...
       placeable.placeRelative(0, 0) // Place children
   ...
}
```

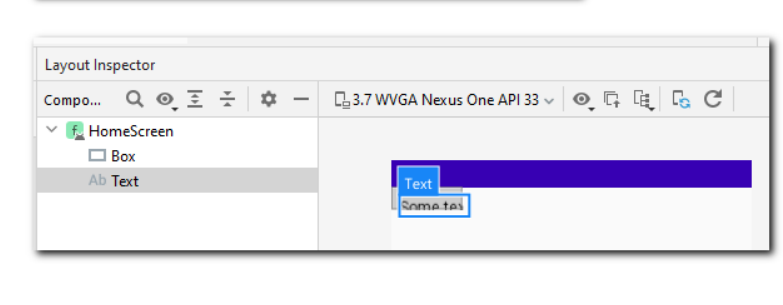
В placeRelative мы дочернему элементу отправляем его будущие координаты. Т.е. если мы представим, что у myLayout есть своя система координат, то Text теперь знает, что ему надо будет размещаться в самом верху слева - (0,0).

Давайте попробуем поместить Text не в (0,0), а в (10,10):

```
fun Modifier.myLayout() = layout { measurable, constraints ->
   val placeable = measurable.measure(constraints) // Measure children
   layout(placeable.width, placeable.height) { // Decide own size
       placeable.placeRelative(10, 10) // Place children
   }
}
```

В итоге текст сместился





Разберем по шагам, что произошло:

Box попросил у myLayout планируемый размер
  myLayout попросил у Text планируемый размер
    Text посчитал свой размер и вернул его в myLayout
  myLayout сообщил Text-у его позицию (10,10)
  myLayout передал в Box свои размеры (такие же, как у Text).
Box поставил себе такой же размер, как у Text

Далее Box поместил myLayout слева вверху внутри себя (т.е. в свои (0,0)), потому что мы не давали никаких других инструкций (align) на этот счет. А myLayout поместил Text правее и ниже, в свои (10,10). В результате мы на экране видим сдвиг.

Почему Box не показывает Text полностью, а обрезает? Потому что Box ничего не знает про этот сдвиг. Он от myLayout получил размеры (такие же, как у Text) и нарисовал себя согласно этим размерам. И сдвинутый Text в них не влез.

Чтобы это исправить, myLayout должен был посчитать свой правильный размер, как размер Text + сдвиг, и передать это значение в Box.

Полученных знаний нам теперь хватит, чтобы создать простые аналоги существующих Modifier операторов: width, offset и padding.

Об этом будет следующий урок. Но вы можете попробовать уже сейчас.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **Тут кажется ошибка и скопировалось не то** — **Александр** 25.06.2023 21:44
Мы в коде указали 200.dp, а в логах получили 300 px. Это не очень удобно, давайте исправим. Будем в коде использовать не dp, а px.

Для этого создадим расширение px:

1

Оно переводит px в dp.

Ответить

# **RE: Тут кажется ошибка и скопировалось не то** — **Dmitry Vinogradov** 27.06.2023 09:11
Пофиксил, спасибо!

Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.